

Tut 10

Anne

HTTP (HyperText Transfer Protocol)

Grundlagen

Design für: Hypertext (HTML)

HTTP -> Applikationsprotokoll (nicht nur Transport)

Statuscode-Klassen:

2xx Erfolg

3xx Weiterleitungen

4xx Client-Fehler

5xx Server-Fehler

CRUD & HTTP-Verben

CRUD	HTTP	Bedeutung
Create	POST	Neue Ressource anlegen
Read	GET	Ressource lesen
Update	PUT	Ressource ersetzen
Delete	DELETE	Ressource löschen

HTML vs. HTTP:

HTML = Markup-Sprache (Inhalt beschreiben)

HTTP = Protokoll (Inhalt übertragen)

HTTP (HyperText Transfer Protocol)

Aufgabe 4 – HTTP

Sie haben in der Vorlesung die essenziellen HTTP-Verben und die Interaktion mit dem Server mittels Statuscodes kennengelernt. Geben Sie für die folgenden beschriebenen Interaktionen das *durch den Client genutzte HTTP-Verb* und den vom Server *zurückgegebenen Statuscode* in einem Browser an.

- a) Sie rufen im Browser eine Webseite auf, die nicht länger existiert.
- b) Sie rufen im Browser eine Webseite auf, der Betreiber hat diese jedoch vor Kurzem auf eine andere Domain umgezogen.
- c) Sie wollen eine Eintragung bei einer Verlosung machen, die dazu genutzte Webseite im Browser ist jedoch völlig überlastet.
- d) Auf einer beliebigen Webseite starten Sie eine Aktualisierung („F5“), da die Seite nur sehr langsam lädt.
- e) Sie aktualisieren die persönlichen Daten Ihres Accounts bei einem Online-Shop.
- f) Ihre eigene Webseite benutzt die Google-Analytics-API, um den Traffic zu überwachen. Bei einem Seitenaufruf wird die ID des aufrufenden Clients zusammen mit dem Zeitpunkt des Aufrufs und der aufgerufenen URL an Google übermittelt.

HTTP Statuscode

Seite existiert nicht mehr

GET

404 Not Found

Seite auf neue Domain umgezogen

GET

301 Moved Permanently

Server komplett überlastet

POST

503 Service Unavailable

Account-Daten aktualisieren

PUT

200 OK / 204 No Content

Analytics-Daten an Google senden

POST (Server als
Client)

200 / 201

Hinweis zu POST

"Möchtest du das Formular erneut senden?"

Idempotente Methoden

GET – immer dasselbe Ergebnis

PUT – überschreibt, kein Duplikat

DELETE – löscht, nochmal löschen = kein Effekt

Nicht idempotent

POST – jeder Call erzeugt eine neue Ressource

Beispiel: 3x "Bestellung absenden" = 3
Bestellungen!

HTML & HTML5

Wichtige HTML-Tags

Tag	Bedeutung
<h1>...<h6>	Überschriften
<p>	Absatz
/	Listen
	Listenelement
 / <i>	Fett / Kursiv

Aufgabe 2 – HTML

Geben Sie den HTML-Code an, der zur folgenden Ausgabe im Browser führt.

Anwendungssysteme

Die Veranstaltung Anwendungssysteme wird an der TU Berlin vom Fachgebiet 3S im Sommer angeboten.

Inhalte sind unter anderem:

- Anwendungsarchitekturen
- Kommunikationsmethoden
- RESTful Webservices

HTML sollten die **Studierenden** aus zwei Gründen beherrschen:

1. Grundlage für das heutige Web
2. Anschauliches Beispiel für eine *Markup-Language*

HTML & HTML5

Aufgabe 3 – HTML5

Nehmen Sie an, Sie besuchen mit Ihrem Browser die Seite `tu.berlin`, die, um zukünftige Zugriffe zu beschleunigen, alle Ihre Benutzerdaten (Noten, Adressdaten, Matrikelnummer, Kontodaten ...) im Web Storage ablegt. Als Nächstes besuchen Sie `chat.openai.com`, um die Lösung für diese HTML5-Aufgabe herauszufinden. Müssen Sie befürchten, dass OpenAI nun Ihre persönlichen Daten kennt? Begründen Sie kurz Ihre Antwort und gehen Sie dabei auch darauf ein, warum wohl dieses Design gewählt wurde.

Same-Origin-Policy:

Jede Domain kann nur auf IHRE eigenen Web-Storage-Daten zugreifen.

Warum?

Privacy: Andere Seiten sehen keine fremden Daten

Konsistenz: Kein Schlüsselkonflikt zwischen Sites

→ OpenAI kann KEINE TU-Berlin-Daten lesen!

3 TU User Pages (2 Punkte)

- Index.html erstellen
- Im Finder cmd k (Mac)
- Pfad eingeben: *smb://tubfs.tu-berlin.de/annenehring*
- Im Browser eingeben: *file:///Users/inesn/Development/index.html*

JavaScript -> Grundlagen

Objektorientierte Skriptsprache für Browser , syntaktisch ähnlich zu Java

Variable & const

```
// let: veränderlich
let x = 10
x = 20 // ok

// const: unveränderlich
const PI = 3.14
// PI = 5 → Fehler!
```

Klasse / Objekt

```
class Haus {
  constructor(höhe, age) {
    this.höhe = höhe
    this.age = age
  }
}
let p = new Haus(15, 2)
```

for-Schleife

```
for (let i = 0; i < 10; i++) {
  console.log(i)
}
```

JavaScript -> Grundlagen

Async

```
//Promise hat immer diese Struktur:
new Promise((resolve, reject) => {
  // irgendwann: resolve() = "fertig, hat geklappt"
  //                reject() = "fertig, ist fehlgeschlagen"
})

//async wenn wir nicht einfrieren wollen
(async function() {
  //Code...
})(); //kann sich selbst aufrufen
```

AJAX: Asynchronous JavaScript and XML

Was ist AJAX?

Daten vom Server laden OHNE die ganze Seite neu zu laden!

1. User klickt "Likes" Button
2. JavaScript schickt kleinen Request im Hintergrund
3. Server antwortet nur mit den neuen Daten (z.B. "42 Likes")
4. JavaScript updated nur den Like-Zähler
5. Seite bleibt gleich, kein Neuladen

Vorteile:

Höhere Interaktivität, Geschwindigkeit

Bekannte Beispiele:

- Google Suche -> Vorschläge erscheinen während du tippst
- Instagram Like -> kein Seitenreload
- Chat-Nachrichten -> kommen rein ohne reload

REST: REpresentational State Transfer

REST = Konvention für URLs & HTTP-Methoden

- URLs beschreiben Ressourcen
- HTTP-Methoden beschreiben Aktionen

Funktion	Methode	URL
neues Buch anlegen	POST	../buch
-> ein bestimmtes Buch lesen	GET	../buch/{id}
Buch löschen	DELETE	../buch/{id}
Buch updaten	PUT	../buch/{id}
Buch zur Ausleihe hinzufügen	PUT	../user/{uid}/ausleihe/{id}/buch/{bid}

... -> steht für Base-Url -> Adresse Server -> Z.B <https://api.meineshop.de/buch>

REST : REpresentational State Transfer

Aufgabe 9 – RESTful Shoppingcart

Gegeben sei eine Shoppingcart-Anwendung, für die Sie ein REST-Interface designen sollen. Die Anwendung verwaltet folgende Entitäten:

- Produkte
- User
- Einkaufswagen
- Wunschlisten

Spezifizieren Sie URL-Schemata und HTTP-Methoden, um ein sinnvolles Interface für die Komponenten zu spezifizieren.

z.B Produkt zur Wunschliste Hinzufügen, User erstellen...

REST: REpresentational State Transfer

Beispiel: RESTful Shoppingcart-Interface

Funktion	URL	Methode
Produkt anlegen	../produkt	POST
Produkt updaten	../produkt/{id}	PUT
Produkt lesen	../produkt/{id}	GET
Alle Produkte	../produkt	GET
Produkt löschen	../produkt/{id}	DELETE
Wunschliste anlegen	../user/{uid}/wunschliste	POST
Wunschliste lesen	../user/{uid}/wunschliste/{id}	GET
Produkt zu Wunschliste	../user/{uid}/wunschliste/{id}/produkt/{pid}	PUT
Produkt aus Liste löschen	../user/{uid}/wunschliste/{id}/produkt/{pid}	DELETE

GitHUB –
Code kommt
später

